AI/ML Residency — Week 6 of 7

Low-Latency Amharic Speech-to-Speech

How the state of the art works, a measured CPU baseline, where the time goes, and what to try next

Biruk Ayalew

25 September 2026

Central question. How can the End-to-End Time to First Audio (T_{TTFA}) of an Amharic speech-to-speech assistant be minimised while keeping recognition, response quality and speech naturalness acceptable?

Constraints for this work. Open-weight, self-hosted models only (no hosted speech or LLM APIs); compute is a 2016 laptop CPU (Intel i5-6300U, 2 cores / 4 threads, AVX2, 15 GB RAM) plus free GPU notebooks (Kaggle, Colab, and others surveyed in Section 5).

How to read the numbers. [M] = measured for this report on the laptop CPU; [L] = from a paper or model card; [P] = projection that still has to be measured.

Everything measured here can be reproduced from `Week-6/smoke/` (Appendix A).

Contents

Summary **2**

1 The problem and the metric **4**
1.1 What the assignment asks for . . . 4
1.2 End-to-End Time to First Audio . . . 4
1.3 How to measure it (protocol used from now on) . . . 4
1.4 What “fast enough” means . . . 5

2 How speech-to-speech systems work **5**
2.1 (a) Cascaded pipelines . . . 5
2.2 (b) Half-cascades: audio-in LLMs . . . 6
2.3 (c) End-to-end speech LLMs . . . 6
2.4 Turn-taking: the stage everyone underestimates . . . 6
2.5 Where production systems spend their time . . . 7

3 What makes Amharic different (for latency and quality) **7**

4 Candidate components (open weights, September 2026) **8**
4.1 Speech recognition (ASR) . . . 8
4.2 Response generation (LLM) . . . 9
4.3 Speech synthesis (TTS) . . . 10
4.4 VAD and turn detection . . . 11
4.5 Orchestration and transport . . . 11

5 Compute available for free **11**

6 Measurements on the laptop CPU **12**
6.1 Setup . . . 12
6.2 Tokenizer cost of Amharic . . . 13
6.3 Speech recognition . . . 13
6.4 Endpointing . . . 14
6.5 Response generation . . . 15
6.6 Speech synthesis . . . 15
6.7 End-to-end baseline . . . 16
6.8 Resources . . . 17

7 Bottleneck analysis **17**
7.1 Ranking on the laptop (best usable configuration) . . . 17
7.2 Projected budgets after optimisation . . . 17

8 Baseline definition and recommended architecture **18**

8.1 The official baseline . . . 18
8.2 Recommended target architecture . . . 19

9 Experiment programme **20**
9.1 Infrastructure (prerequisites) . . . 20
9.2 Endpointing and turn-taking . . . 20
9.3 Speech recognition . . . 21
9.4 Response generation . . . 21
9.5 Speech synthesis . . . 22
9.6 System, orchestration and deployment . . . 23
9.7 The first twelve, in order . . . 23

10 Evaluation protocol **24**
10.1 Test sets . . . 24
10.2 Metrics . . . 25
10.3 Reporting . . . 25

11 Two-week plan **26**

12 Risks, limitations of this report, and mitigations **26**
12.1 Limitations of the measurements in this report . . . 26
12.2 Project risks . . . 27

References **27**

A Reproducing the measurements **28**
A.1 Files . . . 29
A.2 Commands . . . 29
A.3 Building this PDF . . . 29

B End-to-end test turns and transcripts **30**

C Pitfalls found while measuring (for whoever repeats this) **30**

Summary

This report does three things for the Week-6 milestone. It explains how current speech-to-speech systems work and which open models exist for Amharic. It measures every stage of a complete Amharic speech-to-speech pipeline on the laptop CPU. And it turns the measured bottlenecks into a prioritised experiment programme for the rest of Week 6 and all of Week 7.

Baseline and progress (laptop CPU, measured). The frozen baseline B0 (sequential cascade: Whisper-small-WAXAL → Amharic Llama-3.2-1B → MMS-TTS) needs **25.2 s** median from the end of the user's speech to the first response audio. Streaming the LLM into TTS with a 5-word first chunk and switching to an equally accurate CTC recogniser brings this to **5.9 s median (6.8 s p90), a 77% reduction**, with no model made smaller and at equal character accuracy (CER 12.7 vs 13.0). The projected floor is about 1.7–2.7 s on this laptop and 1.1–2.0 s on a free T4 GPU (Section 7).

Key findings.

1. **Only a cascade is buildable for Amharic today.** No open end-to-end speech model outputs Amharic speech. A half-cascade using Gemma 4's audio input is untested for Amharic and is worth one experiment (Section 2).
2. **Tokenization is a latency decision.** Tokens per Amharic word: 12.2 (Whisper, Llama 3.2), 9.1 (Phi-4-mini), 6.5–7.1 (Qwen3/3.5), 3.3 (Gemma 3/4), 1.7 (Amharic-extended Llama). Every token is a sequential decode step in the LLM *and* in Whisper-style ASR (Section 6.2).
3. **CTC beats Whisper on short turns.** Whisper-small and MMS-300M-CTC have the same accuracy (CER 13.0 vs 12.7), but Whisper always encodes a padded 30 s window: its encoder takes 3.8 s for a 5 s question on the laptop, versus 2.4 s for the whole CTC recognition (Section 6.3).
4. **Endpointing is a first-class bottleneck.** Spontaneous Amharic has 7.1 internal pauses per utterance (p90 480 ms). A 500 ms silence rule cuts speakers off 0.53 times per utterance, and about 1 s is needed to be safe. The open semantic turn detector (Smart Turn v3.2) calls 84% of mid-utterance pauses "complete": it does not work for Amharic (Section 6.4).
5. **On CPU the LLM is fast enough but not good enough.** The 1B Amharic model decodes at 14 tok/s (≈8 words/s) with 0.42 s time to first token after prompt caching, but its answers are often wrong. The LLM choice will be decided by quality on the GPU (Section 6.5).
6. **The TTS front-end is a quality gate.** MMS-TTS silently drops digits (46% round-trip CER on a date sentence) and garbles English words (62%). An Amharic text normaliser is required before any quality evaluation (Section 6.6).
7. **Orchestration is worth more than model size.** The same three models went from 25.2 s to 9.2 s by streaming and chunking alone. On two CPU cores the remaining time is ASR finalisation (38%) and first-chunk TTS slowed by LLM contention (37%) (Sections 6.7, 7).

Recommended direction. A streaming cascade with streaming CTC ASR that runs while the user speaks, an Amharic end-of-turn detector with speculative start, a cached and spoken-style LLM prompt, an Amharic text normaliser, and a capped first TTS chunk (Figure 6). It will be optimised in two configurations, laptop CPU and single free T4, and both will be

reported against the frozen baseline.

Next five actions. (1) Record the conversational test set and unify the harness; (2) rerun everything on a Kaggle T4; (3) streaming CTC ASR; (4) text normaliser; (5) CPU contention control and first-chunk tuning. The full backlog of 42 experiments, with decision rules, is in Section 9.

Scope of the numbers End-to-end numbers come from 5 synthetic turns per configuration and ASR numbers from 30 WAXAL utterances. They are reliable for ranking stages and for detecting large effects (2× and more), not as final statistics. Section 12 lists the limitations.

1 The problem and the metric

1.1 What the assignment asks for

The system listens to a user speaking Amharic, understands the request, produces a suitable answer and speaks it back in Amharic. The engineering target is not *whether* this works but *how fast it starts answering*. Accuracy, pronunciation, naturalness, robustness and relevance are quality *constraints*; latency is the *objective*. Week 6 is for building a complete baseline, measuring every stage, and finding the bottlenecks. Week 7 is for removing them, with every change measured against the baseline.

1.2 End-to-End Time to First Audio

Definition. T_{TTFA} is the time from the *acoustic end of the user's speech* (the last voiced frame of the turn) to the *first audible sample of a valid spoken response* at the loudspeaker. Filler sounds ("hmm", a chime) do not count as a valid response.

For a cascaded system, T_{TTFA} decomposes into stages that are either *waiting* or *computing*:

$$T_{\text{TTFA}} = \underbrace{T_{\text{endpoint}}}_{\text{silence wait}} + \underbrace{T_{\text{ASR,final}}}_{\text{finish transcript}} + \underbrace{T_{\text{LLM}\to\text{chunk}_1}}_{\text{prefill + first clause}} + \underbrace{T_{\text{TTS}}(\text{chunk}_1)}_{\text{first audio buffer}} + \underbrace{T_{\text{net}} + T_{\text{play}}}_{\text{transport, device buffer}} \tag{1}$$

Three points follow from Eq. (1). They shape everything that follows.

1. **Only the first chunk matters.** Once the first audio is playing, the rest of the reply only has to be generated faster than it is spoken (real-time factor < 1). A system that streams can hide most of the LLM and TTS cost behind playback.
2. **Waiting counts as much as computing.** T_{endpoint} is pure waiting: the system cannot know the user has finished until some silence has passed. In our measurements (Section 6.4) it is one of the largest terms.
3. **Work done before the endpoint is free.** Anything computed while the user is still speaking (streaming ASR, speculative LLM prefill) comes off T_{TTFA} entirely.

1.3 How to measure it (protocol used from now on)

- **Software timestamps.** Every stage logs `time.perf_counter()` at start and end into one JSON record per turn. The same record also holds the audio duration, the transcript, the reply, token counts and the first-chunk text. The harness in `smoke/e2e_baseline.py` already does this.
- **Reference end-of-speech.** For recorded test turns, the end of speech is the last frame where Silero VAD probability > 0.5. For live tests it is the same, computed offline on the recorded microphone stream.
- **Acoustic ground truth (Week 7).** Play the test turn from one device and record *both* the user audio and the system's loudspeaker output on a second recorder. T_{TTFA} is then the gap between the two waveforms. This captures device buffers and network, which software timestamps miss.
- **Statistics.** Report median, p90 and p95 over at least 30 turns, separately for *warm* (models loaded, caches primed) and *cold* (first request) conditions. Never report a single run.
- **Same inputs for every variant.** All experiments use the same frozen test turns (Section 10), so differences come from the system, not the inputs.

1.4 What "fast enough" means

Human conversation has very short gaps between turns: across ten languages, the typical response gap is around 0–300 ms [1]. Production voice agents built on hosted components in 2026 aim for roughly 0.6–1.2 s end-to-end [2, 3]. For an open-model Amharic system on free hardware, we set practical tiers:

Tier	T_{TTFA} (median)	User perception
Excellent	< 0.8 s	Feels conversational
Good	0.8–1.5 s	Clearly responsive; comparable to hosted voice agents
Acceptable	1.5–3 s	Noticeable pause, still usable
Poor	> 3 s	Users repeat themselves or talk over the system

Realistic goals for this project: **Acceptable on the laptop CPU** and **Good on a free T4 GPU**.

2 How speech-to-speech systems work

There are three families of speech-to-speech conversational systems, and one important question for each: *does it exist for Amharic today, with open weights?*

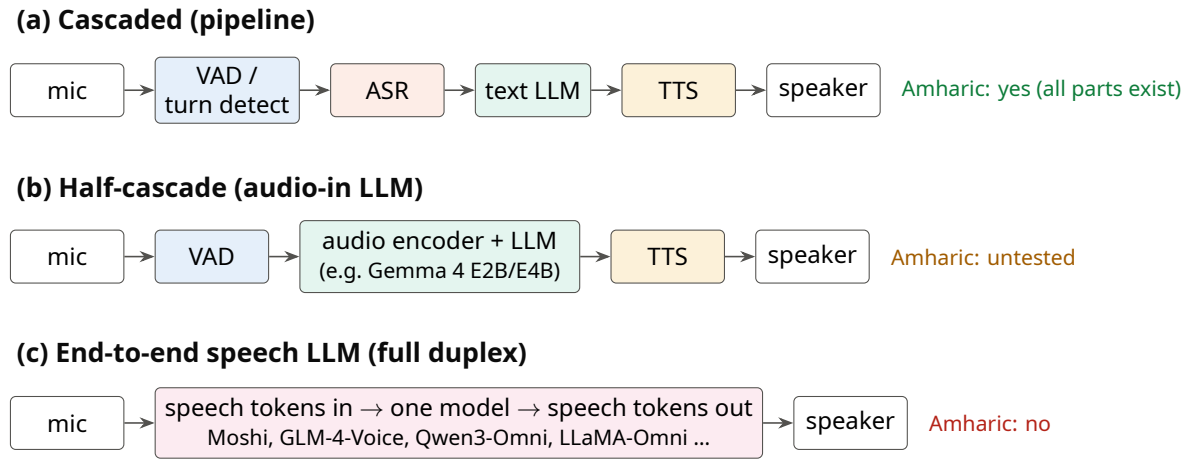


Figure 1. The three system families. Only the cascade can be built for Amharic with open weights today. The half-cascade depends on whether an audio-input LLM understands Amharic speech, which is still to be tested (experiment E-HC-1).

2.1 (a) Cascaded pipelines

A voice-activity detector (VAD) finds speech; a turn detector decides that the user has finished; automatic speech recognition (ASR) turns audio into text; a text LLM writes the reply; text-to-speech (TTS) turns it back into audio. Every component can be swapped independently, and every stage can be measured separately, which is exactly what the assignment requires. The cost is that latency *adds up*, unless the stages overlap.

Sequential vs. streaming cascades. The naive cascade waits for each stage to finish before starting the next. A *streaming* cascade passes partial results forward as soon as they exist:

- **Streaming ASR** emits partial transcripts while the user is still talking, so at the endpoint only the last few hundred milliseconds of audio remain to decode. CTC encoders

do this by processing overlapping chunks. Whisper can be forced into it with the *LocalAgreement* policy [4], at the cost of roughly 3 s lag on GPU. Qwen3-ASR has a native 2-second-chunk streaming mode [5].

- **Streaming LLM** output is standard: tokens arrive one by one. The important quantities are time to first token (TTFT, dominated by *prefill* of the prompt) and time to the first *speakable* chunk, meaning the first clause boundary.
- **Chunked or streaming TTS** synthesises the first clause while the LLM writes the next. Non-autoregressive models (VITS, diffusion) must wait for a whole chunk of text. Codec-LM TTS (e.g. Orpheus with SNAC) can emit audio frames incrementally.

Figure 2 shows why this matters: in the streaming version, T_{TTFA} contains only the *head* of each stage, not its full duration.

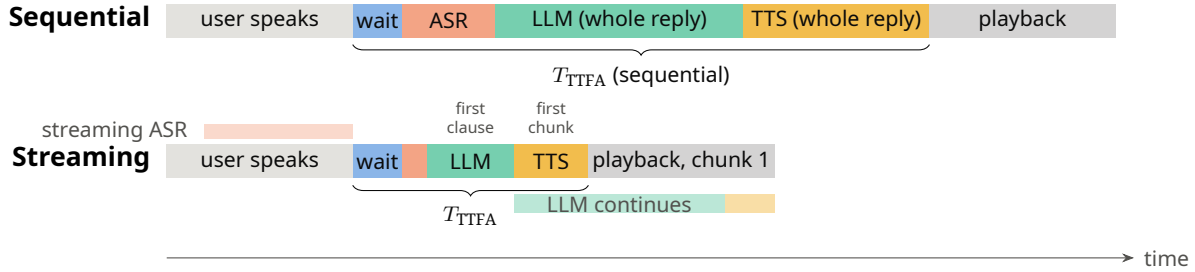


Figure 2. Same components, two orchestrations (schematic, not to scale). Streaming moves ASR work before the endpoint and overlaps LLM and TTS generation with playback. T_{TTFA} then depends on the endpoint wait, the tail of ASR, LLM time to first clause and TTS time for that clause.

2.2 (b) Half-cascades: audio-in LLMs

Some recent LLMs accept audio directly and can answer spoken questions without a separate ASR step. Among open models, Gemma 4 E2B/E4B include a ∼300M-parameter audio encoder, support ASR and speech translation for audio up to 30 s, and were pre-trained on 140+ languages [6]. The model card does not say whether its audio understanding covers Amharic. If it does, the ASR stage disappears, and so does the separate transcript decode that is very slow for Amharic (Section 6.2). This is a high-value, low-cost test (E-HC-1).

2.3 (c) End-to-end speech LLMs

Full-duplex models such as Moshi [7] (about 200 ms practical latency), LLaMA-Omni [8], GLM-4-Voice [9] and Qwen3-Omni [10] model speech tokens directly and can respond while the user is still talking. They represent the lowest latency possible in principle, but none of them produces Amharic speech. SeamlessM4T v2 accepts Amharic speech but outputs Amharic only as *text*, not speech [11]. Training such a model for Amharic is well beyond a two-week budget. **Conclusion: the realistic architecture is a streaming cascade, possibly a half-cascade, and the work is making every head of every stage short.**

2.4 Turn-taking: the stage everyone underestimates

A VAD such as Silero [12] tells speech from silence in under a millisecond per 32 ms frame, but it cannot tell a *thinking pause* from the *end of a turn*. Pure-silence endpointers therefore wait a fixed *hangover* (often 300–800 ms [2]), and that wait is added directly to T_{TTFA}. There are two ways to shorten it:

- **Semantic end-of-turn models**, which judge from prosody or words whether the speaker is done. Smart Turn v3 uses a Whisper-tiny encoder with an 8M-parameter classifier on the last 8 s of audio, and covers 14 languages (not Amharic) [13]. Text-

based detectors run a small LM on the partial transcript.
- **Speculative response**: start ASR finalisation and LLM prefill at the *first* short pause, and throw the work away if the user continues. This hides the endpoint wait whenever the guess is right.

2.5 Where production systems spend their time

Industry latency budgets for (English) voice agents in 2026 put turn detection at 150–400 ms, streaming ASR finalisation at 100–300 ms, LLM TTFT at 250–500 ms and TTS first audio at 100–200 ms, for a total of about 0.6–1.2 s [2, 3]. The two consistently largest items are **turn-taking** and **LLM time to first token**. Prompt caching of a fixed system prompt typically removes 200–400 ms of TTFT [2]. Section 6 tests how these proportions change for Amharic on a laptop CPU.

3 What makes Amharic different (for latency and quality)

Most published latency engineering assumes English. Five properties of Amharic change the picture; the first two are *latency* problems, the rest are *quality* problems that latency optimisations can make worse.

1. The script is expensive for byte-level tokenizers. Amharic is written in the Ge'ez abugida (*fidel*): each character is a consonant+vowel syllable in the Unicode range U+1200–U+137F. UTF-8 stores each character in 3 bytes, and most multilingual BPE vocabularies contain few Ethiopic merges. The result is that one Amharic word costs several times more tokens than one English word, and **every extra token is an extra autoregressive decoding step**, both in the LLM and in any Whisper-style ASR decoder. We measured this directly (Section 6.2): 12.2 tokens per word for Whisper and Llama 3.2, 6.5 for Qwen3, 3.3 for Gemma 3/4, and 1.7 for a Llama with an extended Amharic vocabulary.

2. Spontaneous Amharic speech is full of pauses. In the WAXAL test utterances we found 7.1 internal pauses per utterance, and the 90th percentile pause was 480 ms (Section 6.4). A silence-only endpointer must therefore wait long, or it cuts people off. The open semantic turn detector (Smart Turn v3) was not trained on Amharic, and we found that it does not transfer.

3. The script under-specifies pronunciation. Gemination (doubled consonants) and some vowel distinctions are not written, and several letters are homophones in modern Amharic (ሀ/ሐ/ኀ, ሰ/ሠ, አ/ዐ, ጸ/ፀ). This means that (i) ASR output is judged unfairly unless homophones are folded before computing WER. Gete et al. report that homophone normalisation substantially changes WER [18], and we fold them in all our metrics (`smoke/am_text.py`). It also means that (ii) TTS needs either a pronunciation model or data that implicitly teaches gemination. MMS-TTS romanises input with `uroman` first, which merges ejectives with plain consonants and ignores gemination [19].

4. Numbers, dates, time and calendar. Ethiopia uses its own calendar (currently year 2019 E.C.) and a 12-hour clock that starts at dawn ("ሶስት ሰዓት" is 9 a.m. in international time). An assistant must say these correctly. Our TTS test showed that MMS-TTS *silently drops digits* ("15", "2019"): the round-trip CER of the date sentence was 46% (Section 6.6). A

number-to-words normaliser for Amharic is therefore mandatory before any TTS. It is fast, so it costs no meaningful latency.

5. Code-switching and names. Urban Amharic mixes English words (“meeting”, “Zoom”, “email”). ASR models trained on monolingual Amharic transcribe them in fidel or drop them, and TTS romanisers mispronounce them. Our code-switched sentence had 62% round-trip CER. Ethiopian personal and place names (አበበ በቀለ, ባህር ዳር, ላሊበላ) are partly handled, at 22% CER for the names sentence. The evaluation set must contain all of these (Section 10).

Consequence for design For Amharic, the choice of *tokenizer* is a latency decision, and *text normalisation* is a quality requirement. A model that is fast in English can be several times slower in Amharic just because of how many tokens it has to emit.

4 Candidate components (open weights, September 2026)

This section surveys the open, self-hostable candidates for each stage. For every candidate we record quality evidence, the property that decides its latency, and whether it runs on the laptop CPU or needs a free GPU. Section 6 gives the measurements. WAXAL [14] (Google, CC-BY-4.0, released February 2026) is now the common Amharic benchmark: about 190 h of transcribed, image-prompted *spontaneous* Amharic speech (566 training speakers), a 16 h test set, and a TTS subset. Most numbers below refer to its test set.

4.1 Speech recognition (ASR)

The deciding latency property is the **decoder type**. *Autoregressive* (AR) decoders (Whisper, LLM-based ASR) emit one token per step, and Amharic needs many tokens per word. CTC decoders emit all characters in one parallel pass, so their cost scales with audio length only. The second property is whether the encoder can run **incrementally** while the user speaks.

Model	Params	Decoder	WAXAL WER/CER	Latency-relevant notes
Whisper large-v3, zero-shot	1.55B	AR	254 / 145[L]	Unusable without fine-tuning [15].
MMS-1B, zero-shot	1B	CTC	101 / 85[L]	Unusable zero-shot [15].
Omnilingual ASR 1B, zero-shot	1B	CTC	51.1 / 17.4[L]	1,600+ languages; audio $\leq$40 s; GPU RTF 0.001–0.006 for CTC [16, 15].
Whisper-tiny-WAXAL	39M	AR	41.3 / 16.8[L]	Fast on CPU; fixed 30 s encoder window.
Whisper-small-WAXAL	244M	AR	33.6 / 12.9[L]	Best Whisper option; 3.8 s encoder floor on our CPU[M].
MMS-300M-WAXAL	300M	CTC	37.8 / 13.2[L]	Cost $\propto$ audio length; chunked streaming possible.
OmniASR-CTC-300M-WAXAL	300M	CTC	n/a	fairseq2 checkpoint; same family as Omnilingual.
Ethio-ASR (AfriHuBERT)	94M	CTC	30.95 / –[L]	Small and accurate; checkpoint availability to confirm [17].
Ethio-ASR (MMS-1B)	1B	CTC	26.14 / –[L]	Larger multilingual variant [17].
Ethio-ASR-amharic (w2v-BERT 2.0)	600M	CTC	**22.37** / –[L]	Most accurate open Amharic ASR found; 2.4 GB fp32; T4-friendly.
Qwen3-ASR-0.6B-Amharic	0.6B	AR (LLM)	36.6 / 15.6[L]	Native 2 s-chunk streaming and 92 ms TTFT on GPU [5]; the Qwen tokenizer costs 6.5 tokens/word[M].
Whisper-large-v3-turbo Amharic (Bell)	809M	AR (4 layers)	n/a	Gated. Cheap turbo decoder, but the large encoder is expensive on CPU.
Gemma 4 E2B/E4B audio encoder	$\sim$300M enc.	AR (LLM)	unknown	Half-cascade (Section 2); Amharic audio untested.

Table 1. ASR candidates (model names link to Hugging Face). Most WAXAL numbers are from the WAXAL-NET benchmark [15] or the model cards; WER normalisation differs between sources. Our own numbers (Section 6.3) use one normaliser for all models.

Runtimes for ASR. CTranslate2 / faster-whisper (int8 on CPU, fp16 on GPU), whisper.cpp (supports `--audio-ctx` to shrink the fixed 30 s encoder window for short utterances), ONNX Runtime and OpenVINO for wav2vec2-family CTC models (OpenVINO targets exactly this kind of Intel CPU), sherpa-onnx for streaming CTC/transducer models with a built-in VAD, and vLLM for Qwen3-ASR on GPU.

4.2 Response generation (LLM)

Two properties dominate latency: **time to first token** (prefill, which scales with prompt length and model size and is largely removed by prompt caching) and **tokens per Amharic word** (decode steps). A model that knows Amharic but emits 7 tokens per word is, on the same hardware, about twice as slow per spoken word as one that emits 3.3.

Model	Params	Tok/word[M]	Amharic evidence and latency notes
Llama-3.2-1B-Amharic-Instruct	1.24B	**1.70**	Continued pre-training on Amharic plus an extended vocabulary. Fluent but often factually wrong in our test[M]. 14 tok/s decode on the laptop[M].
Llama-3.2-400M / 180M-Amharic-Instruct	0.4 / 0.18B	–	Weaker. Candidate *draft* models for speculative decoding if their vocabulary matches the 1B model.
Gemma 4 E2B-it	2.3B eff. (5.1B)	3.26	Pre-trained on 140+ languages [6]. Q4 GGUF 3.1 GB. Ships an MTP draft head for speculative decoding. Amharic quality to be tested.
Gemma 4 E4B-it	4.5B eff. (8B)	3.26	Stronger; realistic on a T4 in 4-bit.
Gemma 3 4B / 12B-it	4 / 12B	3.26	Gemma 2 27B was the best open model on most AfroBench languages [20]; 12B fits a T4 in 4-bit.
Qwen3 / Qwen3.5	0.6–4B	6.48 / 7.08	Good general ability, but about 2× the tokens of Gemma per Amharic word.
Qwen3.5-Amharic-4B-thinking (community)	4B	7.08	Amharic fine-tune. "Thinking" must be off: hidden reasoning tokens delay the first spoken word.
Llama 3.2 (base)	1–3B	12.18	Byte-level for Amharic; slow and weak.
Phi-4-mini	3.8B	9.14	Not suitable.

Table 2. LLM candidates. The tokens-per-word column is measured on 3,000 Amharic news sentences (Section 6.2).

Translation cascade (considered and rejected). Translating Amharic→English, answering in English, and translating back (e.g. with NLLB) adds two sequence-to-sequence passes before the first Amharic word exists. That adds about 2× the latency, and the rigid translations hurt naturalness. We keep it only as a quality fallback, not as a latency candidate.

Runtimes for LLMs. llama.cpp (GGUF; CPU and CUDA; prompt cache; speculative decoding including Gemma 4 MTP heads; grammar-constrained output), vLLM and SGLang (GPU; prefix caching; continuous batching for concurrency; fp16 on a T4, which has no bf16), and TensorRT-LLM.

4.3 Speech synthesis (TTS)

The deciding property is **time to first playable audio** for the first chunk. Non-autoregressive models (VITS, diffusion) must synthesise a whole chunk before playing, so a *short first chunk* is the key lever. Codec-LM models (Orpheus + SNAC) can stream audio frame by frame, but they need roughly 80+ codec tokens per second of audio, which means a GPU.

Model	Params	Architecture	Notes
MMS-TTS-amh	36M	VITS (non-autoregressive)	Input romanised with `uroman`; CC-BY-NC-4.0 (non-commercial). RTF 0.57 on the laptop[M]. Drops digits and mispronounces English[M].
OmniVoice-Amharic (Gheero-Leyu fork: link)	613M	Discrete diffusion (Qwen3-0.6B + HiggsAudioV2 codec)	Trained on ∼331 h of Amharic; zero-shot voice cloning from 10 s; Apache-2.0; about 3 GB VRAM on a T4. The base OmniVoice reaches RTF 0.025 on an H100 with 32 steps, and 16 steps is faster [21]. MOS and CER are still "TBD" on the card.
Orpheus-3B-Amharic (+ GGUF)	3B	LLM → SNAC codec tokens	True frame-level streaming; needs a strong GPU for real time; 6.6 GB.
SpeechT5-Amharic	144M	SpeechT5 + HiFi-GAN	Trained on ALFFA; autoregressive spectrogram decoder.
Piper voice trained on WAXAL-TTS	∼20M	VITS (ONNX)	No Amharic voice exists yet. Piper is built for fast CPU inference, so this is a strong candidate for the CPU path (E-TTS-6).

Table 3. TTS candidates. The Gheero-Leyu fork of OmniVoice-Amharic suggests it may already be the organisation's preferred voice. Its speed on a T4 is the most important unknown for Week 7.

4.4 VAD and turn detection

- **Silero VAD** (MIT; 2 MB ONNX): 0.22 ms per 32 ms frame on the laptop[M]. Negligible cost.
- **TEN VAD**: a newer low-latency VAD that claims faster speech-to-silence transitions; worth a direct comparison.
- **Smart Turn v3.2** (BSD-2; 8M params; Whisper-tiny encoder): audio-only semantic end-of-turn for 14 languages. 120 ms per call on our CPU[M]; **does not work on Amharic**[M] (Section 6.4). Its training code is open, so an Amharic fine-tune on WAXAL is feasible (E-EP-3).
- **Text-based end-of-utterance models**: a small LM scores whether the partial transcript is a complete sentence. They need an Amharic model, which could be the 180M Amharic Llama.

4.5 Orchestration and transport

Pipecat (Python; frame-based pipelines; built-in interruption handling, Smart Turn and Silero integration) and **LiveKit Agents** (WebRTC) are the open frameworks used in production voice agents. A plain `asyncio` WebSocket server with 20 ms 16 kHz PCM frames is simpler and makes every millisecond visible. For a single-machine demo, in-process queues avoid all network cost. Our recommendation is to build the measurement harness on plain asyncio, and to adopt Pipecat only if its interruption handling saves real work.

5 Compute available for free

The hardware decides which models are candidates at all, so it is treated as a design variable, not a given. Free-tier limits change often; the numbers below were collected in September 2026 from provider pages and 2026 guides [26, 27, 28] and must be re-checked on sign-up.

Option	Hardware	Limits	Best use in this project
Laptop (ours)	i5-6300U, 2C/4T, AVX2 (no AVX-512 / VNNI), 15 GB RAM, ∼2–5 GB free disk	None; always available	The "edge" configuration. Every measurement in Section 6. Final demo client (microphone and loudspeaker).
Kaggle Notebooks	2×T4 (16 GB each) or 1×P100	∼30 GPU-h/week; sessions capped (9–12 h); internet after phone verification	Main GPU benchmark machine: the same notebook reruns every experiment on the same hardware.
Google Colab (free)	T4 16 GB (when available)	Dynamic quota; idle timeout; ≤12 h sessions	Second GPU, OmniVoice demo notebook, quick checks.
Lightning AI Studio (free tier)	T4/L4 via monthly credits; 1 persistent CPU studio	Monthly credit allowance (sources report 22–80 GPU-h/month)	Persistent environment. Exposing a port for a live demo is simpler than on Kaggle or Colab.
Hugging Face Spaces	CPU basic (free); ZeroGPU needs PRO to host	Cold starts; queueing	Public demo of the CPU pipeline only.
Oracle Cloud Always Free	4 Arm Ampere OCPU, 24 GB RAM	Always on	Optional always-on CPU server (llama.cpp performs well on Arm NEON).
AWS SageMaker Studio Lab	–	Reported closed to new users in July 2026	Not relied on.

Table 4. Free compute options. A T4 (Turing, 65 TFLOPS fp16, 320 GB/s, no bf16) is the realistic GPU target: all fp16/int8 paths must work without bf16.

The network is part of T_{TTFA}. If the models run in a notebook in a US data centre and the microphone is in Addis Ababa, every turn pays at least one round trip for the user's audio and one for the response audio, plus any tunnel overhead (cloudflared, ngrok). This must be measured (experiment E-SYS-4). The design rule is to **co-locate all models on one machine** and to stream audio *while the user speaks*, so that uploading the user's audio overlaps with speaking and only the response direction costs latency.

Two target configurations. We therefore optimise two configurations and report both:

1. **CPU-only (edge)**: the laptop, no network. This is the hardest case and the most honest one, and everything in Section 6 was measured here.
2. **Single T4 (server)**: all models on one Kaggle or Colab T4, with the laptop as a thin audio client. This is the configuration most likely to reach the "Good" tier.

6 Measurements on the laptop CPU

6.1 Setup

- **Hardware:** Intel Core i5-6300U (Skylake, 2 cores / 4 threads, 2.4 GHz, AVX2, no AVX-512/VNNI), 16 GB RAM, no GPU.
- **Software:** Python 3.12, PyTorch 2.13 (CPU), transformers 5.15, CTranslate2 4.8.2 (faster-whisper 1.2.1), ONNX Runtime 1.30, llama.cpp build b11179, uroman 1.3.1, jiwer 4.0.

- **ASR dev data:** 30 utterances from the WAXAL Amharic test split, spread across the split with at most 2 per speaker (19 speakers, both genders, 519 s, median 16.9 s).
- **End-to-end turns:** 5 Amharic questions (Appendix B), synthesised with MMS-TTS and padded with 0.3 s of leading and 1.5 s of trailing silence (2.9–4.4 s of speech).
- **Normalisation:** all WER/CER numbers use `am_text.normalize` (NFC, punctuation removed, homophone rows folded).

6.2 Tokenizer cost of Amharic

We encoded 3,000 random sentences from the Week-4 Amharic news corpus (14.3 words per sentence on average) and the 30 WAXAL transcripts with each candidate's tokenizer.

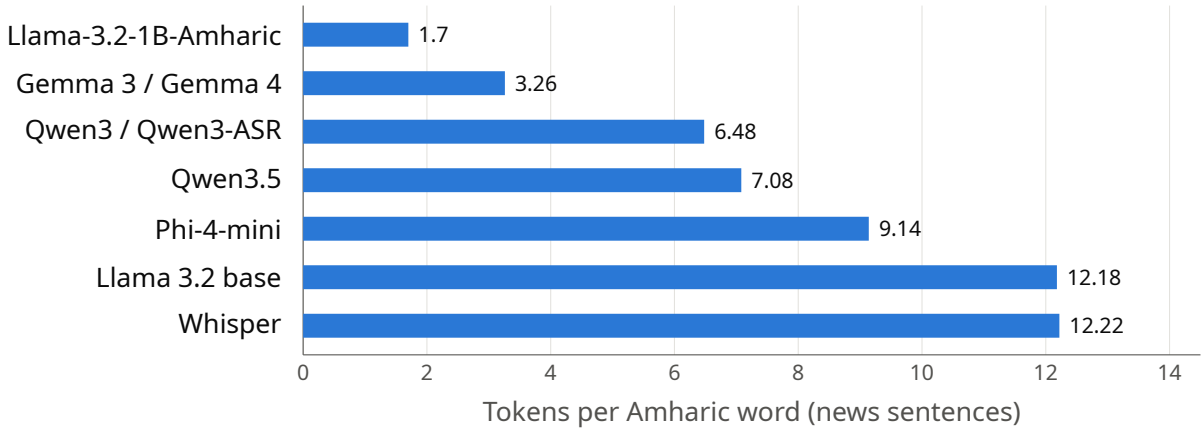


Figure 3. Tokens per Amharic word[M]. English costs 1.1–1.2 tokens per word with all of these tokenizers, so Amharic is 1.4× (extended-vocabulary Llama) to 10.3× (Whisper, Llama base) more expensive. On the WAXAL speech transcripts the numbers are 3–12% lower but in the same order.

Finding 1 — the tokenizer is a latency decision A 30-word Amharic answer costs 51 decode steps with the Amharic Llama, 98 with Gemma, 194 with Qwen3 and 365 with base Llama. The same holds for ASR: the WAXAL Whisper models emit 18.6 tokens per second of audio (Table 5), so a 5 s question needs about 45 sequential decoder steps. This explains why every Whisper variant is slow here, and why CTC is attractive.

6.3 Speech recognition

Model	Runtime	Dec.	WER	CER	Latency med/p90 (s)	RTF	Turn (s)
Whisper-tiny-WAXAL	HF fp32	AR	38.7	18.3	7.82 / 8.99	0.45	–
Whisper-tiny-WAXAL	CT2 int8	AR	38.9	19.0	**3.03** / 3.59	**0.18**	**1.0**
Whisper-small-WAXAL	HF fp32[†]	AR	27.6	11.6	37.9 / 40.9	2.17	–
Whisper-small-WAXAL	CT2 int8	AR	**29.4**	13.0	14.57 / 16.58	0.84	5.6
MMS-300M-WAXAL	HF fp32	CTC	31.4	**12.7**	10.79 / 20.43	0.63	2.4
MMS-300M-WAXAL	+ dynamic int8	CTC	33.7	13.4	15.56 / 19.88	0.91	–

Table 5. ASR on 30 WAXAL test utterances (median 16.9 s) on the laptop, 4 threads, greedy decoding[M]. HF = Hugging Face transformers. "Turn" is the median ASR time inside the end-to-end runs (2.9–4.4 s questions). [†]Only 8 utterances (too slow for the full set).

Finding 2 — what the ASR numbers say **(a)** CTranslate2 int8 is 2.6× faster than HF fp32 for Whisper, with no accuracy loss. **(b)** Whisper-small and MMS-300M-CTC are equally accurate (CER 13.0 vs 12.7), but on short turns CTC is 2.3× faster (2.4 s vs 5.6 s). The reason is that Whisper always encodes a padded 30 s window: for a 5 s question the encoder alone takes 3.8 s (small) or 0.55 s (tiny), and the decoder adds 1.9 s (38 ms per token × 45 tokens)[M]. **(c)** PyTorch dynamic int8 *slows* the CTC model down on this CPU (RTF 0.63 → 0.91) and costs 2.3 WER points: without VNNI, int8 matrix multiplies do not pay off. Try ONNX Runtime / OpenVINO instead (E-ASR-6).

Pitfall found: faster-whisper truncates Amharic faster-whisper caps generation at 448 positions, and CTranslate2's Whisper `generate()` returned only `max_length`/2 tokens. As a result, *every* transcript stopped at 224 tokens (≈18 Amharic words), and WER looked like 57% instead of 39%. We bypass it by calling `generate(max_length=880)` directly (`smoke/asr_bench.py`). Any team using faster-whisper for Amharic should check for this.

6.4 Endpointing

Silero VAD (ONNX, 1 thread) costs **0.22 ms per 32 ms frame**[M]: 0.8% of real time, which is negligible. The problem is deciding *when* the user is done. Inside the 30 WAXAL utterances we found 7.1 internal pauses per utterance, with pause lengths p50 = 160 ms, p90 = 480 ms and p99 = 892 ms[M].

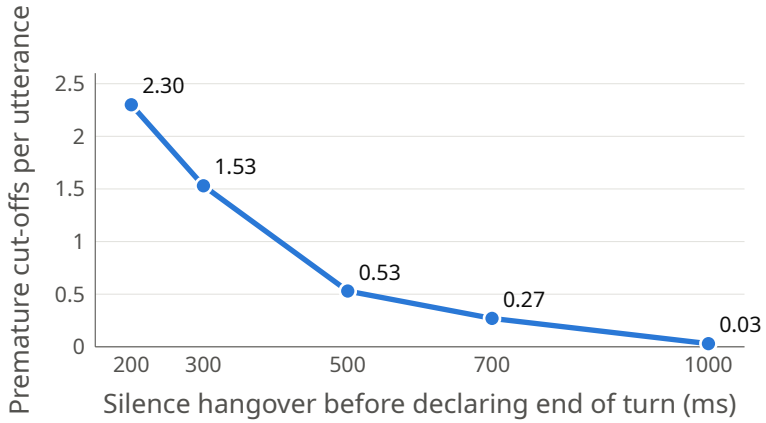


Figure 4. How often a pure-silence endpointer would cut a speaker off mid-utterance, as a function of the hangover[M]. Data: 30 spontaneous WAXAL utterances, averaging 17 s. Shorter conversational questions will have fewer pauses (E-EP-1).

We then tested **Smart Turn v3.2** (CPU ONNX) at the true end of each utterance and at each of 69 internal pauses of at least 200 ms. It returned P(complete) with a median of 0.94 at true ends, *but 0.95 at mid-utterance pauses*: 84% of internal pauses were classified as "turn complete"[M]. Each call took 120 ms on this CPU (the authors report 12 ms on modern CPUs [13]).

Finding 3 — endpointing is a first-class bottleneck With silence only, a safe hangover for spontaneous Amharic is 700–1000 ms of pure waiting. That is already more than an entire English voice agent's LLM budget. The off-the-shelf semantic detector does not understand Amharic. An Amharic end-of-turn model (E-EP-3, E-EP-4) and speculative turn-ending (E-EP-5) are therefore among the highest-value experiments, and they help equally on CPU and GPU.

6.5 Response generation

Model: `rasyosef/Llama-3.2-1B-Amharic-Instruct`, GGUF Q4_K_M (825 MB), llama.cpp server, context 2,048, temperature 0.3.

llama-bench	**2 threads**	**4 threads**
Prompt processing, 128 / 512 tokens (tok/s)	46.0 / 45.3	45.2 / 45.5
Generation, 64 tokens (tok/s)	**14.2**	12.8

Streaming, 5 questions (median)	**cold**	**warm**
Prompt tokens / prefill (ms)	61 / 1,490	11 / 396
Time to first token (ms)	1,504	**416**
Time to first clause (ms)	2,584	**1,378**
Tokens per word of replies	1.6	

Table 6. LLM on the laptop CPU[M]. "Warm" = system prompt cached in the KV cache, question not cached. Hyper-threading hurts decoding (2 threads beat 4).

Finding 4 — the LLM is fast enough on CPU, but not good enough At 14 tokens/s and 1.6 tokens per word, the 1B Amharic model generates about 8 words/s, faster than speech, and prompt caching cuts TTFT from 1.5 s to 0.42 s. **Quality is the limit**: replies are fluent Amharic but often wrong or off-topic. Examples: it claimed coffee is "4–5% salt", put "pizza" in a coffee answer, and after an ASR error answered that Ethiopia's capital is "Manhattan". It also ignores the "one or two sentences" instruction (up to 82 words). Relevance, not speed, decides the LLM (E-LLM-2).

6.6 Speech synthesis

Sentence	**Words**	**Audio (s)**	**Synthesis (ms)**	**RTF**	**Round-trip CER**
greeting	4	2.94	1,536	0.52	5.0%
capital	6	4.13	2,246	0.54	4.2%
weather	12	6.91	3,920	0.57	12.8%
museum hours (Ethiopian time)	12	5.76	3,380	0.59	16.7%
names and places	18	10.75	6,195	0.58	22.0%
digits and date	9	3.81	2,121	0.56	**46.2%**
code-switching	12	5.42	3,094	0.57	**62.5%**

Table 7. MMS-TTS (VITS, 36M parameters, 16 kHz) on the laptop, 2 threads, median of 3 runs[M]. Round-trip CER = CER of Whisper-small-WAXAL transcribing the synthesised audio (lower is more intelligible). 4 threads gave no gain (RTF 0.59). Loading takes 9.8 s, and the first call is slower (lazy initialisation).

Finding 5 — TTS: short first chunks and text normalisation VITS cost is linear in output length (RTF $\approx$ 0.57), so the first audio of a 4-word chunk takes about 1.5 s and a whole 40-word reply about 14 s. Chunking is therefore mandatory on CPU. The digits "15" and "2019" are silently dropped (the romanised vocabulary has no digits), and English words come out unintelligible ("Zoom" $\rightarrow$ ዐጾም). A normaliser is required (E-TTS-3).

6.7 End-to-end baseline

The harness (`smoke/e2e_baseline.py`) streams each user turn through Silero VAD with a 700 ms hangover, transcribes up to the endpoint, streams the LLM, and synthesises in one of three modes: **A** sequential (whole reply), **B** first clause (first clause mark after $\geq$2 words), **C** first clause *or* 5 words, whichever comes first. In B and C the LLM keeps generating in parallel while the first chunk is synthesised, so the two compete for the same 2 cores, as they would in a real system. LLM: 2 threads; TTS: 2 threads.

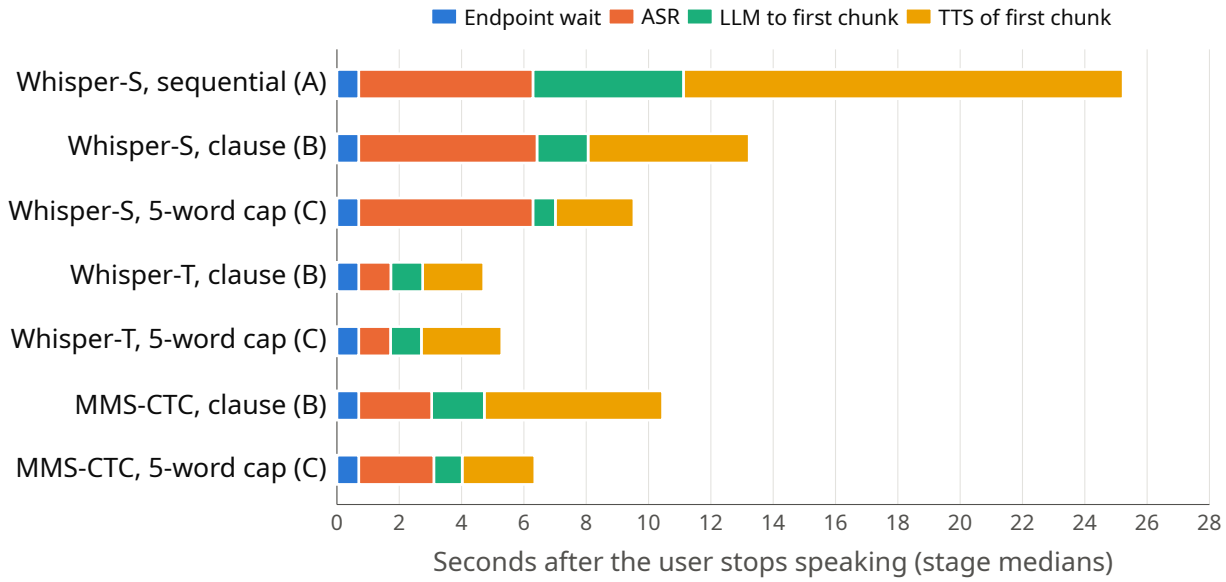


Figure 5. Where the time goes, per configuration (medians of each stage over 5 turns)[M]. Whisper-S / Whisper-T = Whisper small / tiny WAXAL (CT2 int8); MMS-CTC = MMS-300M-WAXAL (fp32); LLM = Amharic Llama 1B Q4_K_M; TTS = MMS-TTS. Median T_{TTFA} per row is given in Table 8.

ASR	Mode	T_{TTFA} median (s)	p90 (s)	min–max (s)	Largest stage
Whisper-small	A sequential	25.2	46.3	7.2–50.6	TTS of whole reply (14.1 s)
Whisper-small	B clause	13.0	17.5	9.4–18.7	ASR (5.7 s), TTS (5.2 s)
Whisper-small	C 5-word cap	9.2	10.6	8.7–10.8	ASR (5.6 s)
Whisper-tiny	B clause	**4.4**	9.2	4.2–11.7	TTS (2.0 s)
Whisper-tiny	C 5-word cap	5.4	6.4	4.0–6.4	TTS (2.6 s)
MMS-300M CTC	B clause	10.8	15.2	4.5–16.4	TTS (5.7 s)
MMS-300M CTC	C 5-word cap	**5.9**	**6.8**	4.4–6.8	ASR (2.4 s), TTS (2.3 s)

Table 8. End-to-end T_{TTFA} on the laptop CPU, 5 turns per row[M]. Whisper-tiny is the fastest, but on these turns its CER is 22.0%, against 15.6% for MMS-300M-CTC and 14.7% for Whisper-small (Appendix B). That matches the WAXAL ranking, so MMS-300M-CTC with the 5-word cap is the best *usable* CPU configuration so far.

Finding 6 — streaming gives 2.7×, the CTC swap another 1.6× With the *same three models* on the *same laptop*, sequential processing takes 25.2 s median and streaming with a capped first chunk takes 9.2 s. Swapping Whisper-small for a CTC model of equal accuracy brings it to 5.9 s (4.3× better than the sequential baseline). Clause-only chunking (B) has a long tail: when the model's first sentence is long, synthesising it while the LLM is still decoding takes 5–11 s. The cap (C) removes that tail (p90 15.2 → 6.8 s).

6.8 Resources

Component (loaded, after one inference)	RSS	Load time
Whisper-tiny-WAXAL, CT2 int8	129 MB	<1 s
Whisper-small-WAXAL, CT2 int8	551 MB	<1 s
MMS-300M-WAXAL CTC, fp32	1,602 MB	6.7 s
MMS-TTS (VITS)	167 MB	9.8 s (incl. uroman)
llama.cpp server, 1B Q4_K_M, 2k context	1,457 MB	2.9 s
Full CPU pipeline (CTC + LLM + TTS)	≈3.2 GB	13 s

The whole pipeline fits comfortably in 16 GB of RAM; memory is not a constraint on the CPU path. Compute (2 physical cores shared by three models) is the constraint.

7 Bottleneck analysis

7.1 Ranking on the laptop (best usable configuration)

For the MMS-300M-CTC, 5-word-cap configuration (median T_{TTFA} 5.9 s), the stage medians are:

Stage	Median (s)	Share	Nature of the cost
ASR after endpoint (offline CTC over the whole turn)	2.41	38%	compute; avoidable by streaming
TTS of the first chunk (under LLM contention)	2.33	37%	compute + contention
LLM to first 5 words (warm cache)	0.91	14%	compute (prefill + ∼8 tokens)
Endpoint wait (700 ms hangover)	0.70	11%	*pure waiting*

(The shares are of the sum of stage medians, 6.35 s, which differs slightly from the median T_{TTFA}.)

Interpretation.

1. **ASR finalisation** is the largest term, and almost all of it is avoidable: the audio was available while the user spoke. Streaming CTC (E-ASR-4) should reduce it to roughly one chunk (0.2–0.4 s on this CPU[P]).
2. **First-chunk TTS** costs 2.3 s, of which about 0.9 s is contention. The first chunks are about 2.5 s of audio, which would take about 1.4 s on an idle CPU (RTF 0.57). While the LLM keeps decoding in parallel, the effective RTF rises to 0.8–1.2[M]. Contention control (E-SYS-2) and a faster runtime or voice (E-TTS-5/6) attack it.
3. **The LLM** is not the latency problem on this hardware once the prompt is cached; it is the *quality* problem.
4. **The endpoint wait** is small in this configuration only because the other terms are large. On a GPU it becomes the dominant term.

7.2 Projected budgets after optimisation

The table below is a *projection*[P] built from our CPU measurements, the literature numbers in Section 4, and the expected effect of the experiments. It is the hypothesis Week 7 has to confirm or reject, not a result.

Stage	CPU now[M]	CPU target[P]	How
Endpoint wait	0.70	0.30–0.45	Amharic end-of-turn model + speculative start (E-EP-3/4/5)
ASR after endpoint	2.41	0.25–0.40	Streaming CTC during speech; ONNX/OpenVINO (E-ASR-4/6)
LLM to first chunk	0.91	0.6–0.8	Caching (done), spoken prompt, pinned cores (E-LLM-4/7)
TTS first chunk	2.33	0.5–1.0	Contention control, ONNX VITS or Piper voice (E-SYS-2, E-TTS-5/6)
T_{TTFA}	**5.9**	**1.7–2.7**	"Acceptable" tier on a 2016 laptop

Stage	T4 target[P]	Basis
Endpoint wait	0.30–0.45	Same as CPU (waiting does not get faster on a GPU)
ASR after endpoint	0.05–0.15	CTC encoders run at RTF ∼0.001–0.006 on GPU [16]; the last streaming chunk only
LLM to first chunk	0.3–0.5	Gemma 4 E2B/E4B in 4-bit, cached prompt, ∼16 tokens for the first 5 words
TTS first chunk	0.2–0.5	MMS-TTS on GPU, or OmniVoice with 8–16 steps on a short chunk [21]
Network (laptop ↔ notebook)	0.2–0.4	One response-direction trip plus the tunnel; to be measured (E-SYS-4)
T_{TTFA}	**1.1–2.0**	"Good" tier, with the endpoint wait and the network now dominant

Where Week 7 should spend its time On the CPU: ASR streaming and TTS contention first (together ≈75% of T_{TTFA}). On the GPU: the endpoint wait and the network, because compute stops being the bottleneck. Both paths share the Amharic end-of-turn work. That is why E-EP-3/5 are P0 even though they save less on the CPU today.

8 Baseline definition and recommended architecture

8.1 The official baseline

The assignment asks for improvement measured against a baseline, so the baseline must be fixed now and never edited later.

Baseline B0 (frozen 25 Sep 2026). A straightforward, non-streaming cascade using the most accurate components that run on the laptop: Silero VAD with a 700 ms hangover → Whisper-small-WAXAL (CT2 int8) → Llama-3.2-1B-Amharic-Instruct Q4_K_M (llama.cpp, 2 threads, cached system prompt) → MMS-TTS of the whole reply. **Measured T_{TTFA}: 25.2 s median, 46.3 s p90** (5 synthetic turns; to be re-measured on the recorded conversational set, E-INF-2). The same configuration on a Kaggle T4 is baseline B0-GPU (E-INF-3).

Already measured improvements over B0 (same laptop):

Change (cumulative)	T_{TTFA} median	vs B0	Keep?
B0: sequential	25.2 s	–	reference
+ LLM streaming, TTS of the first clause (mode B)	13.0 s	−48%	keep streaming; first-chunk rule changed below
+ first chunk capped at 5 words (mode C)	9.2 s	−64%	keep
+ ASR: Whisper-small → MMS-300M CTC (equal CER)	**5.9 s**	**−77%**	keep (current best)
Alternative: ASR → Whisper-tiny	5.4 s	−79%	reject: CER 19 vs 13 (WAXAL), 22 vs 16 (turns)
Tried: dynamic int8 on the CTC model	slower	–	reject: RTF 0.63 → 0.91

8.2 Recommended target architecture

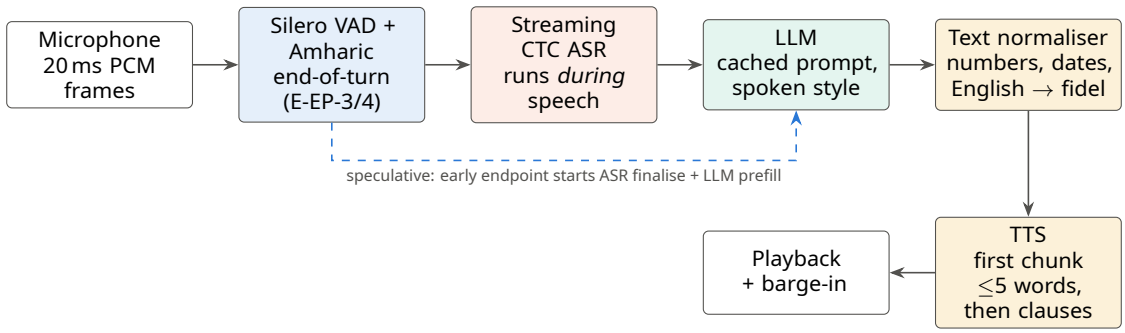


Figure 6. Target architecture: a streaming cascade in which every stage starts on partial input. The dashed path is speculative turn ending: work starts at the first long pause and is discarded if the user continues.

CPU path	ONNX/OpenVINO CTC ASR; llama.cpp Amharic Llama 1B (2 threads, pinned); MMS or Piper VITS (2 threads, priority over the LLM).
GPU path (T4)	Ethio-ASR or MMS CTC in fp16; Gemma 4 E2B/E4B in 4-bit; OmniVoice-Amharic with 8–16 steps; all on one GPU, with the laptop as a thin audio client.

Why a cascade. It is the only family that exists for Amharic with open weights today (Section 2). It can be measured stage by stage, and it lets a different model be chosen for the CPU and GPU configurations without changing the orchestration. The half-cascade (Gemma 4 audio-in) is kept as an alternative branch and will be tested once (E-HC-1). If Gemma 4 understands Amharic speech well, it removes the ASR stage on the GPU path.

Why CTC ASR. Equal accuracy to Whisper-small, cost proportional to audio length, no Amharic token tax, and natural streaming.

Why the first chunk is capped. Every configuration measured shows that the *length* of the first spoken chunk, not the LLM, sets the TTS cost. A 5-word cap removed the long tail (p90 15.2 → 6.8 s). E-TTS-4 will tune the cap against naturalness.

9 Experiment programme

This is the backlog for the rest of Week 6 and all of Week 7. Every experiment follows the same contract:

Contract. (1) State a hypothesis with an expected effect on T_{TTFA} in milliseconds. (2) Change *one* thing relative to the current best configuration. (3) Rerun the frozen test set (Section 10) with the shared harness and report median, p90 and the stage breakdown. (4) Apply the *decision rule*: **keep** the change if the median T_{TTFA} improves by at least 10% or 200 ms (whichever is smaller) *and* no quality guard is violated. The guards are ASR CER +2 points absolute; TTS round-trip CER +3 points; LLM relevance rating -0.3 on the 1–5 scale; false cut-off rate +5 points. (5) Log the experiment, including failed ones, in `DECISIONS_LOG.md`.

Priorities: **P0** = do first, large expected gain or a prerequisite; **P1** = strong candidate; **P2** = stretch or insight only. Hardware: **C** = laptop CPU, **G** = free T4. Expected gains are [P] projections from Section 7 and must be measured.

9.1 Infrastructure (prerequisites)

ID	Experiment and hypothesis	Measure / decision	HW	Pri	Est.
E-INF-1	Unify the smoke scripts into one harness: one JSON record per turn with every stage timestamp, transcripts, reply, token counts, CPU/RAM/VRAM; one command reruns all variants on the frozen test set.	Reproducibility: two consecutive runs agree within 5% median.	C,G	P0	–
E-INF-2	Record the frozen *conversational* test set (Section 10): real people asking real questions, not image descriptions.	$\geq$60 turns, $\geq$8 speakers, all stress categories covered.	–	P0	–
E-INF-3	Kaggle T4 notebook that runs the same harness, giving the GPU baseline with no code changes.	Laptop vs T4 table for every stage.	G	P0	–
E-INF-4	Acoustic loopback: play the turn from device 1, record user audio and system output on device 2, and compute T_{TTFA} from waveforms.	Gap between acoustic and software T_{TTFA} $<$100 ms, or explain it.	C	P1	–

9.2 Endpointing and turn-taking

The endpoint wait is pure waiting and is the one cost that no amount of GPU removes.

ID	Experiment and hypothesis	Measure / decision	HW	Pri	Est.
E-EP-1	Sweep silence hangover 300–1200 ms on the conversational test set. Our monologue data suggest $\geq$700 ms is needed, but questions are probably shorter and more fluent.	False cut-off rate vs added latency curve; choose the knee.	C	P0	–
E-EP-2	Silero VAD vs TEN VAD: time from the true speech offset to the VAD “silence” decision.	Offset delay (ms), false alarms in noise.	C	P1	50–150 ms

E-EP-3	Fine-tune Smart Turn v3 on Amharic. Positives are true utterance ends; negatives are internal pauses $\geq$200 ms (WAXAL gives thousands of both). The measured baseline marks 84% of mid-turn pauses as "complete".	Mid-pause false-complete $<$10%, end recall $>$90%; allows 200–300 ms hangover.	G (train), C	P0	300–500 ms
E-EP-4	Text-side completion cue: Amharic is verb-final (SOV), so a partial transcript ending in a finite verb or copula (ነው, ናት, ነበር, ይችላል, question forms) is likely complete. Combine with the acoustic score.	Same metrics as E-EP-3; cost $<$5 ms.	C	P1	100–300 ms
E-EP-5	Speculative turn end: at the first pause $\geq$250 ms, finalise ASR and start LLM prefill and first-chunk generation; commit at the real endpoint, discard if the user resumes.	T_{TTFA} gain; wasted-compute rate; no audio may play before commit.	C,G	P0	up to the full hangover
E-EP-6	Adaptive hangover: shorter after a completed question (rising pitch, question particle), longer after a hesitation (እ..., lengthened vowel).	False cut-offs at a lower mean wait.	C	P2	100–200 ms

9.3 Speech recognition

ID	Experiment and hypothesis	Measure / decision	HW	Pri	Est.
E-ASR-3	CTC models on *short* turns: MMS-300M-WAXAL, Ethio-ASR w2v-BERT (600M), OmniASR-CTC-300M-WAXAL. Unlike Whisper, cost scales with audio length, with no fixed 30 s window.	WER/CER and latency on 2–8 s turns; int8 on CPU, fp16 on T4.	C,G	P0	2–4 s on CPU
E-ASR-4	Streaming CTC: run the encoder on 1 s chunks with left context *while the user speaks*, so only the last chunk remains after the endpoint.	Post-endpoint ASR latency (target $<$150 ms CPU); WER penalty vs offline.	C,G	P0	most of ASR
E-ASR-5	Shrink Whisper's fixed 30 s encoder window for short turns (whisper.cpp `--audio-ctx`, or truncated positional context).	Encoder time (3.8 s $\rightarrow$?) and WER on turns $<$8 s.	C	P1	1–3 s on CPU
E-ASR-6	ONNX Runtime / OpenVINO int8 export of the CTC finalist (OpenVINO targets Intel CPUs like ours).	RTF vs PyTorch dynamic int8.	C	P1	30–60%
E-ASR-7	Qwen3-ASR-0.6B-Amharic with native 2 s-chunk streaming on the T4.	Post-endpoint latency, WER vs CTC finalist.	G	P1	–
E-ASR-8	Robustness of finalists: additive noise at 20/10/5 dB SNR, speed 0.85$\times$/1.15$\times$, names, numbers, code-switching.	WER/CER per condition; no latency change expected.	C,G	P0	quality
E-ASR-9	Name and term biasing: CTC beam search with a small KenLM n-gram plus a hotword list (Ethiopian names, places, local terms).	Entity accuracy vs added decode time.	C	P2	quality
E-ASR-10	Whisper with an Amharic-extended vocabulary (fine-tune): cuts decode steps about 4–7$\times$ (Section 6.2).	Decoder ms/utterance, WER.	G	P2	1–2 s

9.4 Response generation

ID	Experiment and hypothesis	Measure / decision	HW	Pri	Est.
E-LLM-2	Model ladder on the T4: Amharic-Llama-1B, Gemma 4 E2B and E4B, Gemma 3 4B/12B, Qwen3.5-Amharic-4B (thinking off).	TTFT, time-to-first-clause, tok/s, plus 1–5 human ratings (relevance, fluency, correctness) on 30 questions.	G	P0	quality
E-LLM-3	Prompt caching of the system prompt and dialogue history (already 1.5 s → 0.42 s TTFT on CPU[M]).	TTFT cold vs warm on every model.	C,G	P0	0.3–1.1 s
E-LLM-4	Spoken-style prompting: “answer first in one short sentence”; a short first clause; hard `max_tokens`.	Words in the first clause, time-to-first-clause, rating.	C,G	P0	0.3–1 s
E-LLM-5	Quantisation sweep: Q8_0, Q6_K, Q4_K_M, IQ4_XS, Q3_K_M for each finalist.	tok/s, TTFT, rating.	C,G	P1	20–40%
E-LLM-6	Speculative decoding: Gemma 4 MTP head (published with the GGUFs); a 180M/400M Amharic Llama as draft for the 1B (if the vocabularies match).	tok/s and time-to-first-clause; acceptance rate.	C,G	P1	1.5–2× decode
E-LLM-7	Thread and core allocation: 2 vs 4 threads (2 is faster[M]); pin the LLM and TTS to different physical cores.	Contention-free first-chunk TTS.	C	P0	0.5–2 s on CPU
E-LLM-8	LLM sensitivity to ASR errors: feed transcripts at WER 0/15/30/45% and rate the answers.	Rating vs WER curve; tells how much ASR accuracy may be traded for speed.	G	P1	decision
E-LLM-9	LoRA fine-tune of the fastest acceptable model on short, factual, spoken-style Amharic QA to reduce verbosity and hallucination.	Rating and reply length.	G	P2	quality

9.5 Speech synthesis

ID	Experiment and hypothesis	Measure / decision	HW	Pri	Est.
E-TTS-3	Amharic text normaliser before TTS: digits → words, Ethiopian dates and times, ዓ.ም., abbreviations, and English words → Ge’ez transliteration (ዙም, ሚቲንግ).	Round-trip CER on the digits sentence (46% now) and the code-switch sentence (62%); cost <5 ms.	C	P0	quality
E-TTS-4	First-chunk policy: clause boundary vs cap at 3/5/7 words vs minimum 1 s of audio.	T_{TTFA} vs naturalness (MOS) of the first chunk.	C,G	P0	0.5–2 s CPU
E-TTS-5	MMS-TTS runtime: ONNX Runtime / OpenVINO export on CPU; fp16 on T4.	RTF (0.57 now on CPU).	C,G	P1	2×
E-TTS-6	Train a Piper (VITS, ONNX) Amharic voice on one WAXAL-TTS speaker; Piper is designed for fast CPU inference.	CPU RTF (target <0.15), MOS, round-trip CER.	G (train), C	P1	1–2 s CPU
E-TTS-7	OmniVoice-Amharic on the T4: diffusion steps 32/16/8, fp16, first-chunk length.	First-chunk latency, RTF, MOS, round-trip CER.	G	P0	quality
E-TTS-8	Orpheus-3B-Amharic streaming (SNAC frames) on the T4.	Time to first audio frame; sustain RTF <1?	G	P2	–
E-TTS-9	Pipelined synthesis of chunks 2…n during playback, with an underrun check.	Zero playback gaps at RTF under contention.	C,G	P1	reliability

E-TTS-10	Cached openers: prompt the LLM to begin with one of a few fixed acknowledgements (እሺ፣ ጥሩ ጥያቄ ነው።) whose audio is pre-synthesised. Confirm with the instructors whether this counts as “valid response”.	T_{TTFA} if allowed.	C,G	P2	0.5–1 s

9.6 System, orchestration and deployment

ID	Experiment and hypothesis	Measure / decision	HW	Pri	Est.
E-SYS-1	Streaming orchestration (done for the first baseline): sequential → clause streaming → capped first chunk.	25.2 → 13.0 → 9.2 s measured with whisper-small (Section 6.7).	C	done	–
E-SYS-2	Contention control on CPU: pause LLM decoding while the first chunk is synthesised, or raise TTS priority.	First-chunk TTS time under load (1.4–2.5 s, vs 1.0 s alone).	C	P0	0.5–1 s
E-SYS-3	Model residency and warm-up: all models loaded once, dummy inference at start-up, no lazy initialisation.	Cold vs warm T_{TTFA}.	C,G	P0	first turn
E-SYS-4	Network cost: laptop client ↔ Kaggle/Colab/Lightning server via tunnel; WebSocket vs WebRTC.	RTT distribution and its share of T_{TTFA}.	C+G	P0	decision
E-SYS-5	Hybrid placement: streaming ASR on the laptop during speech, and only the text goes to the GPU for LLM+TTS.	T_{TTFA} vs all-on-GPU.	C+G	P1	upload time
E-SYS-6	Concurrency on one T4: 1/2/4/8 simultaneous sessions (vLLM batching for the LLM).	T_{TTFA} p90 vs sessions; VRAM.	G	P1	report
E-SYS-7	Barge-in: stop playback and cancel generation when the user speaks.	Stop latency $<$200 ms.	C,G	P2	UX
E-HC-1	Half-cascade: Gemma 4 E2B/E4B with Amharic *audio* input: (a) transcribe, (b) answer directly from audio.	CER; answer rating; T_{TTFA} with no ASR stage.	G	P0	ASR stage

9.7 The first twelve, in order

Ordered by expected T_{TTFA} gain per hour of work, with prerequisites first:

1. **E-INF-2** + **E-INF-1**: record the conversational test set; unify the harness.
2. **E-INF-3**: rerun everything on a Kaggle T4, giving the second baseline.
3. **E-ASR-3** + **E-ASR-4**: CTC ASR, then streaming CTC. This removes the largest CPU term (ASR, 61% of T_{TTFA} in mode C).
4. **E-TTS-3**: text normaliser (a quality gate; needed before any MOS test).
5. **E-SYS-2** + **E-LLM-7**: CPU contention control and core pinning.
6. **E-TTS-4**: first-chunk policy.
7. **E-EP-1**: hangover sweep on real questions.
8. **E-LLM-2** + **E-LLM-3** + **E-LLM-4**: model ladder with caching and spoken-style prompts (quality decision).
9. **E-HC-1**: Gemma 4 audio-in check (one notebook cell; could remove ASR).
10. **E-TTS-7**: OmniVoice-Amharic speed and quality on the T4.
11. **E-EP-5**: speculative turn end.
12. **E-EP-3**: Amharic Smart Turn fine-tune.

10 Evaluation protocol

Latency is the objective; the protocol below makes sure it is not bought with an unusable system.

10.1 Test sets

1. **Dev set (already built).** 30 WAXAL test utterances from 19 speakers, both genders, 519 s in total (`smoke/audio/waxal_amh_test/`). These are spontaneous image descriptions, which is good for ASR accuracy and pause statistics but not typical of questions.
2. **Conversational turn set (to record, E-INF-2).** At least 60 spoken questions, 2–8 s each, from at least 8 speakers (mixed gender, age and regional accent), recorded on laptop and phone microphones at 16 kHz. Every turn has a reference transcript and a reference "good answer" note. Planned coverage:

Category	Turns	Example
Everyday questions	12	ቡና እንዴት ይፈላል?
Ethiopian personal names	8	አበበ በቀለ ማን ነው?
Places and geography	8	ከአዲስ አበባ ወደ ባህር ዳር ስንት ኪሎ ሜትር ነው?
Numbers, dates, Ethiopian time	8	መስከረም 15 ቀን 2019 ምን ቀን ነው?
Local terminology (health, agriculture, services)	8	የጤና መድን እንዴት ይገኛል?
Amharic–English code-switching	8	የ bank account እንዴት ልክፈት?
Fast and slow speakers	4+4	same prompts, deliberately paced

3. **Noise variants.** Each turn mixed with café, street and fan noise at 20, 10 and 5 dB SNR (public-domain noise or noise recorded locally), for ASR robustness (E-ASR-8).
4. **Held-out final set.** A further 30 turns from speakers *not* used during tuning, recorded once and only used for the final report (the assignment requires previously unseen speech).

10.2 Metrics

Metric	Definition
T_{TTFA} (primary)	Last voiced user frame $\rightarrow$ first audible response sample; median, p90, p95; warm and cold.
Stage breakdown	T_{endpoint}, T_{ASR} after the endpoint, LLM TTFT, LLM time-to-first-chunk, TTS first-chunk time, network.
ASR WER / CER	After NFC normalisation, punctuation removal and homophone folding (`am_text.py`); reported per category.
ASR RTF	Processing time / audio duration; plus post-endpoint finalisation latency for streaming ASR.
LLM TTFT, tok/s	From the server timings; tokens per Amharic word of actual replies.
Response quality	1–5 ratings by two native speakers for relevance, correctness and fluency on 30 questions; blind to configuration.
TTS intelligibility	Round-trip CER: transcribe synthesised audio with the best ASR (already implemented).
TTS naturalness	5-point MOS from $\geq$5 native listeners on 10 sentences per system (names, numbers, code-switch included).
False cut-off rate	Share of turns where the system started answering before the user had finished.
Resources	Peak RSS, CPU utilisation, VRAM (`psutil`, `nvidia-smi`); model load time.
Concurrency	T_{TTFA} p90 at 1/2/4/8 simultaneous sessions (T4 only).

10.3 Reporting

Every result table states hardware, software versions, model files (repository and quantisation), thread counts, the test set, and whether the numbers are warm or cold. Each final claim is backed by a JSON file in `results/` and the script that produced it.

11 Two-week plan

A day-by-day plan for one person. Work items refer to experiment IDs in Section 9. Days already covered by this report are marked **done**.

Day	Work	Output
Week 6 — baseline, research, bottlenecks		
Wed 23 – Thu 24	Read the brief; survey architectures and Amharic models. **done**	Sections 2–5
Fri 25	CPU smoke tests of every stage; first end-to-end baseline; this report. **done**	Sections 6–8, `smoke/`
Sat 26	E-INF-2: record the conversational test set; E-INF-1: unify the harness.	Frozen test set, harness v1
Sun 27	E-INF-3: Kaggle T4 notebook running the same harness; E-TTS-3: text normaliser.	GPU baseline table
Mon 28	E-ASR-3: CTC models on short turns (CPU and T4); E-HC-1: Gemma 4 audio-in check.	ASR decision
Tue 29	E-LLM-2/3/4: LLM ladder with caching and spoken prompts; start quality ratings.	LLM shortlist
Wed 30	E-TTS-7: OmniVoice-Amharic on T4; MMS vs OmniVoice MOS round 1. Week-6 checkpoint: baseline T_{TTFA} (CPU and T4) and bottleneck ranking.	Week-6 report v2
Week 7 — optimisation and final system		
Thu 1	E-ASR-4: streaming CTC during speech; E-SYS-2 + E-LLM-7: contention control.	Largest CPU gain
Fri 2	E-TTS-4: first-chunk policy; E-EP-1: hangover sweep; E-SYS-4: network measurement.	Endpoint setting
Sat 3	E-EP-5: speculative turn end; E-LLM-6: speculative decoding / MTP.	
Sun 4	E-EP-3: Amharic Smart Turn fine-tune on Kaggle (training runs in the background).	
Mon 5	Integrate winners into the final system; live demo path (laptop client $\leftrightarrow$ T4).	Final system
Tue 6	Final evaluation on the held-out set: latency, WER/CER, MOS, ratings, resources, concurrency.	Final numbers
Wed 7	Final report and defence: baseline vs final, what worked, what failed, limitations.	Submission

12 Risks, limitations of this report, and mitigations

12.1 Limitations of the measurements in this report

- **Small samples.** ASR used 30 utterances; the end-to-end runs used 5 turns per mode. The numbers show which stage dominates and by how much, but they are not final statistics.
- **Synthetic user turns.** The end-to-end test questions were synthesised with MMS-TTS because no conversational recordings exist yet. ASR errors on them are therefore not representative, which is why E-INF-2 is first.
- **Monologue pause statistics.** WAXAL utterances are picture descriptions; questions are probably more fluent, so the endpoint numbers are a pessimistic bound.
- **Old CPU.** The i5-6300U (2016) has no AVX-512/VNNI, so int8 kernels gain less than on a modern CPU. Every CPU result is a lower bound for “laptop” performance in general.

- **Contention.** The laptop was also running an editor and a browser; we used medians to limit the effect, but absolute numbers can shift by roughly 10%.

12.2 Project risks

Risk	Mitigation
No open LLM that is both fast and good in Amharic	Decide on the T4 ladder (E-LLM-2) early; accept a larger model on the GPU and win the time back with caching, short first clauses and speculative decoding.
Free-GPU session limits and queueing during the demo	Keep the CPU path demo-ready; keep Kaggle and Colab notebooks identical; pre-download weights to a Kaggle dataset.
Network latency from Ethiopia to US-hosted notebooks	Measure it (E-SYS-4); stream user audio during speech; fall back to the CPU demo.
MMS-TTS licence is non-commercial	Fine for the assignment. For production, OmniVoice-Amharic (Apache-2.0) or a Piper voice trained on WAXAL (CC-BY-4.0).
Disk space on the laptop ($\sim$2 GB free)	Store large checkpoints on Kaggle; keep only int8/Q4 builds locally; the Xet cache (`~/.cache/huggingface/xet`, 9.4 GB) can be cleared.
Speculative tricks start speaking too early	Never play audio before the committed endpoint; count false starts as a guarded metric.
Quality regressions hidden by latency wins	The decision rule's quality guards; blind human ratings on a fixed set.

References

[1] T. Stivers et al. Universals and cultural variation in turn-taking in conversation. *PNAS* 106(26):10587–10592, 2009.

[2] WebRTC.ventures. The Voice AI Latency Budget: Where Every Millisecond Goes. Sept. 2026. https://webrtc.ventures/2026/09/voice-ai-latency-budget/

[3] Deepgram. Voice Agent Architecture: STT, LLM, and TTS Pipeline Guide 2026. https://deepgram.com/learn/voice-agent-architecture-stt-llm-tts-pipeline-design

[4] D. Macháček, R. Dabre, O. Bojar. Turning Whisper into Real-Time Transcription System. IJCNLP-AACL 2023 (demo). arXiv:2307.14743.

[5] Qwen Team. Qwen3-ASR Technical Report. arXiv:2601.21337, 2026.

[6] Google DeepMind. Gemma 4 model card. https://ai.google.dev/gemma/docs/core/model_card_4; https://huggingface.co/google/gemma-4-E2B-it

[7] A. Défossez et al. Moshi: a speech-text foundation model for real-time dialogue. arXiv:2410.00037, 2024.

[8] Q. Fang et al. LLaMA-Omni: Seamless Speech Interaction with Large Language Models. arXiv:2409.06666, 2024.

[9] A. Zeng et al. GLM-4-Voice: Towards Intelligent and Human-Like End-to-End Spoken Chatbot. arXiv:2412.02612, 2024.

[10] Qwen Team. Qwen3-Omni Technical Report. arXiv:2509.17765, 2025.

[11] Seamless Communication et al. Seamless: Multilingual Expressive and Streaming Speech Translation. arXiv:2312.05187, 2023; language table at https://huggingface.co/facebook/seamless-m4t-v2-large.

[12] Silero Team. Silero VAD. https://github.com/snakers4/silero-vad

[13] Pipecat / Daily. Smart Turn v3 (model, benchmarks, training code). https://github.com/pipecat-ai/smart-turn; https://huggingface.co/pipecat-ai/smart-turn-v3

[14] Google Research. WAXAL: a large-scale open resource for African language speech technology. 2026. https://huggingface.co/datasets/google/WaxalNLP

[15] WAXAL-NET: Finetuned Edge ASR Across 19 African Languages. arXiv:2606.02375, 2026. Models at https://huggingface.co/waxal-benchmarking

[16] Omnilingual ASR Team (Meta). Omnilingual ASR: Open-Source Multilingual Speech Recognition for 1600+ Languages. arXiv:2511.09690, 2025. https://github.com/facebookresearch/omnilingual-asr

[17] B. M. Abdullah et al. Ethio-ASR: Joint Multilingual Speech Recognition and Language Identification for Ethiopian Languages. arXiv:2603.23654, 2026. https://huggingface.co/badrex/Ethio-ASR-amharic

[18] D. K. Gete et al. Whispering in Amharic: Fine-tuning Whisper for Low-resource Language. arXiv:2503.18485, 2025.

[19] V. Pratap et al. Scaling Speech Technology to 1,000+ Languages. arXiv:2305.13516, 2023. TTS model: https://huggingface.co/facebook/mms-tts-amh

[20] J. O. Ojo et al. AfroBench: How Good are Large Language Models on African Languages? Findings of ACL 2025. arXiv:2311.07978.

[21] k2-fsa. OmniVoice. https://github.com/k2-fsa/OmniVoice; Amharic fine-tune: https://huggingface.co/african-low-resource/omnivoice-amharic

[22] Y. Worku (rasyosef). Llama-3.2-1B-Amharic-Instruct. https://huggingface.co/rasyosef/Llama-3.2-1B-Amharic-Instruct

[23] ggml-org. llama.cpp. https://github.com/ggml-org/llama.cpp

[24] SYSTRAN. faster-whisper (CTranslate2). https://github.com/SYSTRAN/faster-whisper

[25] Pipecat. Open-source framework for voice and multimodal agents. https://github.com/pipecat-ai/pipecat

[26] Kaggle. Efficient GPU usage tips. https://www.kaggle.com/docs/efficient-gpu-usage

[27] Google. Colab FAQ. https://research.google.com/colaboratory/faq.html

[28] Lightning AI. Pricing and free tier. https://lightning.ai/pricing/

A Reproducing the measurements

A.1 Files

Path (under `Week-6/smoke/`)	Purpose
`fetch_waxal.py`	Downloads 30 WAXAL Amharic test clips through the datasets-server rows API (no 900 MB shards) and writes `audio/waxal_amh_test/manifest.jsonl`.
`download_models.py`	Downloads only the inference files of the tested models.
`am_text.py`	Amharic normalisation for WER/CER (NFC, punctuation, homophone folding).
`sentences_am.py`	TTS test sentences and user questions.
`tokenizer_fertility.py`	Tokens per Amharic word for eight tokenizers.
`asr_bench.py`	ASR accuracy and latency (`hf-whisper`, `ct2-whisper`, `hf-ctc` backends).
`whisper_split.py`	Whisper encoder vs decoder time on short turns.
`endpoint_bench.py`	Silero VAD cost, pause statistics, Smart Turn on Amharic.
`llm_bench.py`	llama.cpp server streaming: TTFT, first clause, cold/warm cache.
`tts_mms_bench.py`, `tts_roundtrip.py`	MMS-TTS latency and intelligibility (ASR round trip).
`e2e_baseline.py`	End-to-end harness, modes A/B/C, per-stage timing.
`resources.py`, `summarize.py`	Memory footprint; collects all results into `results/summary.json`.
`results/*.json`	Raw outputs behind every number in this report.

A.2 Commands

```
cd Week-6 && python3 -m venv .venv && .venv/bin/pip install -r smoke/requirements.txt
cd smoke
../.venv/bin/python fetch_waxal.py && ../.venv/bin/python download_models.py
../.venv/bin/python tokenizer_fertility.py
# Whisper -> CTranslate2 int8
../.venv/bin/ct2-transformers-converter --model <whisper-small-waxal-amh snapshot> \
    --output_dir models/whisper-small-waxal-amh-ct2-int8 --quantization int8 \
    --copy_files tokenizer.json preprocessor_config.json
../.venv/bin/python asr_bench.py --backend ct2-whisper \
    --model models/whisper-small-waxal-amh-ct2-int8 --label whisper-small-waxal_ct2-int8
../.venv/bin/python asr_bench.py --backend hf-ctc --model <mms-300m-waxal-amh snapshot> \
    --label mms300m-waxal_ctc-fp32
../.venv/bin/python endpoint_bench.py
../.venv/bin/python llm_bench.py --gguf <Llama-3.2-1B-Amharic-Instruct.Q4_K_M.gguf> \
    --label llama32-1b-am-q4km_t2 --threads 2
../.venv/bin/python tts_mms_bench.py && ../.venv/bin/python tts_roundtrip.py
../.venv/bin/python e2e_baseline.py --asr <mms-300m snapshot> --asr_backend ctc \
    --gguf <gguf> --label cpu_mms300mCTC-llama1B-mms \
    --llm_threads 2 --hangover_ms 700 --modes BC
python3 summarize.py
```

llama.cpp is the prebuilt CPU release `b11179` in `smoke/bin/`. The Week-6 virtual environment reuses PyTorch and transformers from the Week-4 environment through a `.pth` file to save about 1 GB of disk; `requirements.txt` lists everything needed for a clean install.

A.3 Building this PDF

`cd Week-6/report && lualatex main && lualatex main`. The laptop's TeX Live 2023 lacked `luaotfload`, so the matching TL-2023 packages `luaotfload`, `lualibs` and `lua-uni-algos` were installed into `~/texmf` (no root needed). Ge'ez text switches to Noto Serif Ethiopic automatically through `babel`'s `onchar` mechanism.

B End-to-end test turns and transcripts

Mode C runs, hangover 700 ms. S = Whisper-small-WAXAL, T = Whisper-tiny-WAXAL, CTC = MMS-300M-WAXAL. CER over the five turns: S 14.7%, T 22.0%, CTC 15.6% [M].

Reference	የኢትዮጵያ ዋና ከተማ ማን ናት?
S / T / CTC	የትዮግያ ዋና ከተማማማ ናት / የትዮገያ ዋና ከተማ ማናት ነው። / የትዮጵያ ዋና ከተማ ማን ናት
LLM reply (CTC run)	አዲስ አበባ
Reference	ነገ ወደ ባህር ዳር ለመሄድ ምን ማድረግ አለብኝ?
S / T / CTC	ነገረ ወደ ባህር ዳር ለመሄድ ምን አድረጋለብኝ / ነገረ ወደ ባህር ዳለመሄድ ምን ማዘረጋለለል አለደኛ / ነገር ወደ ባህር ዳ ለመሄድ ምን ማድረግ አለድኙ
LLM reply (CTC run)	ውሃ ውስጥ ገብተህ ሞቃት ውሃ ውስጥ ከገባህ በኋላ ውሃው እንዲሞቅ አድርግ። (off-topic)
Reference	ለጉንፋን ምን ማድረግ ይሻላል?
S / T / CTC	ለጉንፋን ምን ማድረግ ይሻላል / ለቡንፋን ምን ማድረግ ይሻላል / ለጎንፋን ምን ማጥረግ ይሻላል
Reference	ስለ ላሊበላ አብያተ ክርስቲያናት በአጭሩ ንገረኝ።
S / T / CTC	ስለ ላሊበላ አብያተ ክርስቲያናት ባችሁ ነገርኝ / ስለላሊበላ አብያተ ክርስቲያናት ባቸው ነገረኝ። / ስለላሊመላ አብያተ ክርስቲያናት ባችሉ ንጋረኝ ነው።
Reference	ቡና እንዴት ይፈላል?
S / T / CTC	እና እንዴት ይፈላል / ውና እንዴት ይፈላል / ቡና እንዴት ይፈላል
LLM reply (CTC run)	በቡና ላይ ያለው የጎንዮሽ ጉዳት ቡና መጠጣት እና ማኘክ ነው. (fluent, not an answer)

The questions were synthesised with MMS-TTS, so these transcripts say little about accuracy on real voices. They are included to show how ASR errors propagate into the LLM's answer.

C Pitfalls found while measuring (for whoever repeats this)

1. **faster-whisper truncation**: Amharic transcripts are silently cut at 224 tokens (Section 6.3).
2. **"Warm cache" that is too warm**: if the benchmark repeats the *same* question, the whole prompt is cached and TTFT looks like 80 ms. Prime only the system prompt (`llm_bench.py` does).
3. **Sentence boundaries**: the Amharic LLM often ends sentences with an ASCII "." rather than ።, so the clause detector must accept both.
4. **Empty first chunks**: list markers such as "1." romanise to nothing and crash VITS; require at least two words.
5. **Hyper-threading**: 2 threads beat 4 for llama.cpp decoding and for VITS on this 2-core CPU.
6. **Disk**: the Hugging Face Xet chunk cache (`~/.cache/huggingface/xet`) grows to about 10 GB; set `HF_XET_CHUNK_CACHE_SIZE_BYTES=0` when disk is tight.